電腦圖學 Homework #2
A Moving Robot

黃俊源 01257128　2026-11-03

1　使用者操作說明

1.1　基本單位定義

世界坐標以公尺為單位，即 1單位 = 1公尺。模擬固定以 60 Hz 更新，所以 1 Tick = $\frac{1}{60}$ 秒。

1.2　按鍵

表 1 列出機器人與相機的操作方式。移動方向以機器人的視線為準：W、S 控制前後移動，A、D 控制左右平移。站立時可以走路、跑步或跳躍；潛行時只能慢走或跳躍；坐下時不能移動。姿勢切換需要 0.5 秒，切換途中同樣不能移動。走路、潛行與跑步的速度分別為 1.2、0.6 與 3 m/s。機器人只有在站立或潛行且雙腳著地時才能跳躍，跳躍高度為 0.5 m。

Table 1: 操控機器人與相機的按鍵

按鍵	功能
W / S	往視線方向前進 / 後退
A / D	往左 / 右平移
左 Shift	按住時跑步
空白鍵	跳躍 0.5 m；需站立或潛行且著地
1 / 2 / 3	切換為站立 / 潛行 / 坐下
左鍵拖曳	左右轉動、上下俯仰視角
Q / E	往左 / 右轉動視角
Tab	切換相機模式（第 1.3 節）

畫面左上角的 HUD 會顯示幀率（FPS）、模擬頻率（TPS）、視線的 yaw 與 pitch，以及機器人的位置和速度。

1.3 相機模式

相機固定以機器人的頭部為關注點（focus），但會依模式改變位置（eye）。按下 Tab 可以在三種模式之間切換：

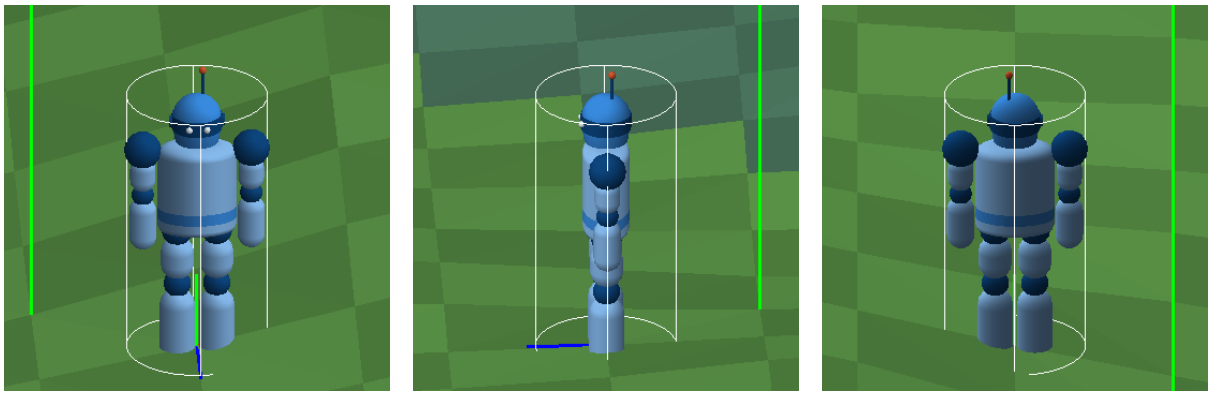
- **正面**：相機在機器人前方，機器人看向相機，轉動視角時仍會帶動機器人轉身。
- **背後**（預設）：相機在機器人後方，轉動視角時機器人跟著轉身，適合一般移動。
- **自由**：機器人維持原本的視線，相機可以繞著機器人轉，適合從側面或正面觀察動作。

2 機器人模型

2.1 外觀與尺寸

模型尺寸以「像素」表示，1px = $\frac{1}{16}$公尺。機器人高 28 px（1.75 m）、寬 14 px（0.875 m）、厚 5 px（0.3125 m）。

機器人由頭、脖子、軀幹，以及左右上臂、前臂、大腿與小腿組成，共 11 個部位（圖 1）。各部位都有自己的局部坐標系（LCS），原點設在關節上；$+y$ 軸朝上、$+z$ 軸朝向機器人前方、$+x$ 軸朝左。



(a) 正面 (b) 側面 (c) 背面

Figure 1: 機器人的外觀

每個部位都是一組三角形網格（mesh）。我先用直線、圓弧與 Bézier 曲線在 $+xy$ 半平面畫出 2D 剖面，再透過 `MeshBuilder` 將剖面繞 y 軸旋轉成旋轉體（lathe）。部位的平移、旋轉與縮放則使用 4×4 齊次矩陣處理。

模型一開始只使用純色，各三角形的顏色完全相同，外觀看起來很平。為了保留簡單的配色，又能看出表面起伏，我依三角形的法向量調整亮度。作法參考 Inigo

Quilez 的 YouTube 教學：先計算單位法向量 $\mathbf{n}$ 與太陽方向 $\mathbf{l}$ 的內積。直接使用 $\max(0, \mathbf{n}\cdot\mathbf{l})$ 會讓所有背光面變成相同的暗色，因此我改將內積從 $[-1,1]$ 映射到 $[0,1]$ 後再平方：

$$b(\mathbf{n}) = \left(\tfrac{1}{2}\,\mathbf{n}\cdot\mathbf{l} + \tfrac{1}{2}\right)^2, \qquad s(\mathbf{n}) = \min\!\left(1,\; 0.32 + 0.68 \times 0.9 \times \frac{b(\mathbf{n})}{b(\hat{\mathbf{y}})}\right).$$

亮度倍率 s 的下限設為 0.32，避免背光面全黑；平地（$\mathbf{n} = \hat{\mathbf{y}} = [0,1,0]^\top$）的倍率校正為 0.9，因此朝向太陽的斜面仍可比平地更亮。

2.2 場景圖

機器人的場景圖如圖 2。每個節點是一根骨頭（bone），記錄它的父節點、關節在父節點座標系中的位置（pivot），以及可轉動的角度範圍。根節點 root 本身沒有關節，只帶有一個隨姿勢改變的平移量 rootOffset，用來決定髖部離地的高度。軀幹的原點在髖部中央，雙腿從髖部往下、雙臂從肩膀往下延伸，脖子接在軀幹頂端，頭再接在脖子上。

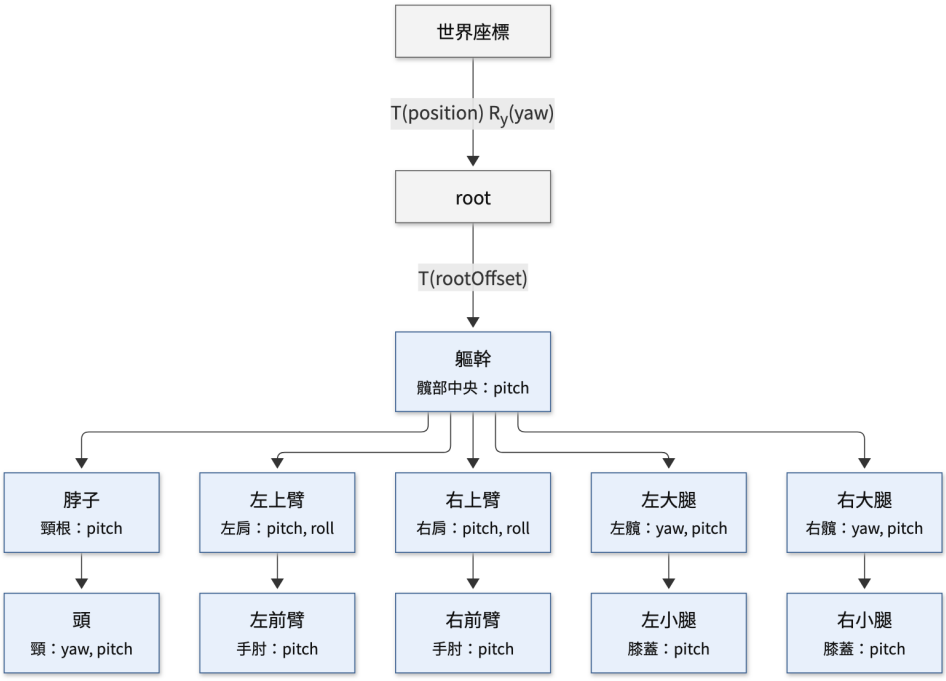


Figure 2: 機器人的場景圖，節點下方為連接父節點的關節與其可轉動的軸

2.3 關節自由度

每個關節最多有 yaw（繞 y 軸）、pitch（繞 $-x$ 軸）、roll（繞 z 軸）三個旋轉自由度；角度範圍上下限相同的軸視為鎖住，不算一個自由度。骨頭在靜止姿勢時面向 $+z$，pitch 取 $-x$ 為轉軸，正值就是往前抬起，與視線的 pitch 方向一致。各關節的自由度與角度範圍如表 2，角度範圍以左側為準，右側的 yaw 與 roll 範圍左右對稱、

正負相反。四肢共有 12 個自由度，加上軀幹的 1 個與頭部的 2 個，機器人身上共有 15 個旋轉自由度；另外整隻機器人在世界中還有位置 (x, y, z) 與朝向 yaw 共 4 個自由度。

Table 2: 關節的自由度與角度範圍（「—」表示鎖住）

關節	yaw	pitch	roll	自由度
軀幹（髖部中央）	—	$-45° \sim 45°$	—	1
脖子（頸根）	—	—	—	0
頭（頸）	$-70° \sim 70°$	$-60° \sim 50°$	—	2
上臂（肩膀）×2	—	$-60° \sim 200°$	$0° \sim 170°$	2
前臂（手肘）×2	—	$0° \sim 145°$	—	1
大腿（髖）×2	$0° \sim 45°$	$-30° \sim 120°$	—	2
小腿（膝蓋）×2	—	$-140° \sim 0°$	—	1

2.4 階層繪製

階層模型的變換順序是先移到關節位置，再套用關節旋轉；所有子節點都會繼承這個變換。程式用相同順序計算骨頭 i 的矩陣：

$$\begin{cases} M_{\text{root}} = M_0 = T(\text{rootOffset}), \\ M_i = M_{\text{parent}(i)} T(\text{pivot}_i) R_y(\text{yaw}_i) R_{-x}(\text{pitch}_i) R_z(\text{roll}_i), i \neq \text{root}. \end{cases}$$

骨頭依加入順序儲存，而且父節點一定在子節點之前，因此從頭走過陣列一次就能算完所有矩陣，不需要遞迴。計算矩陣前，各角度會先限制在表 2 的範圍內，避免姿勢與動畫疊加後超出關節活動範圍。

繪製時先以 `glTranslatef`、`glRotatef` 移到機器人的位置與朝向，再對每根骨頭 `glPushMatrix`、`glMultMatrixf`(M_i)、畫出網格、`glPopMatrix`。矩陣採用 OpenGL 的 column-major 排列，可以直接交給 `glMultMatrixf`。

這組矩陣不只用來繪圖。眼睛在頭部上是一個標記點（marker），將它的局部座標乘上頭部矩陣 M_{head} 後，就能得到世界座標，相機再以該位置為焦點。骨架矩陣的計算放在不依賴 OpenGL 的 `world` 模組中，模擬端與繪圖端使用的是同一份結果。

3 動作與控制

3.1 從輸入到動作

主迴圈透過 `glutTimerFunc` 以約 60 Hz 更新。每次更新會讀取輸入、更新機器人狀態、進行物理模擬，最後重畫畫面。

按鍵狀態不會直接改變位置，而是先轉成 `Intent`。其中記錄前後左右的移動量、跑步與跳躍狀態、要切換的姿勢及視線方向。機器人每個 tick 依下列順序處理：

1. 身體朝向設為視線的 yaw，轉彎因此由視角控制
2. 處理姿勢切換的請求，並推進轉換的進度
3. 依目前的姿勢決定目標速度，交給物理模擬器計算位移與碰撞（第 4 節）
4. 以實際移動的距離推進走路的步伐

關節角度等到繪製前才由姿勢與目前步伐算出。輸入端只記錄玩家想做的動作；速度、姿勢限制與碰撞仍由模擬端判斷。

3.2 姿勢切換

機器人有站立、潛行與坐下三種姿勢（圖 3）。每種姿勢包含一組關節角度和 rootOffset：

- **站立**：所有關節為 0°，髖部高度等於腿長 $L = 13$ px。
- **潛行**：軀幹前傾 30°、脖子回抬 30° 讓視線維持水平，大腿往前抬 60°、膝蓋彎 60°；髖部降到 $\frac{\sqrt{3}}{2}L$。
- **坐下**：大腿往前抬 90° 並向外張開 15°，髖部降到大腿厚度的一半，正好坐在地上。

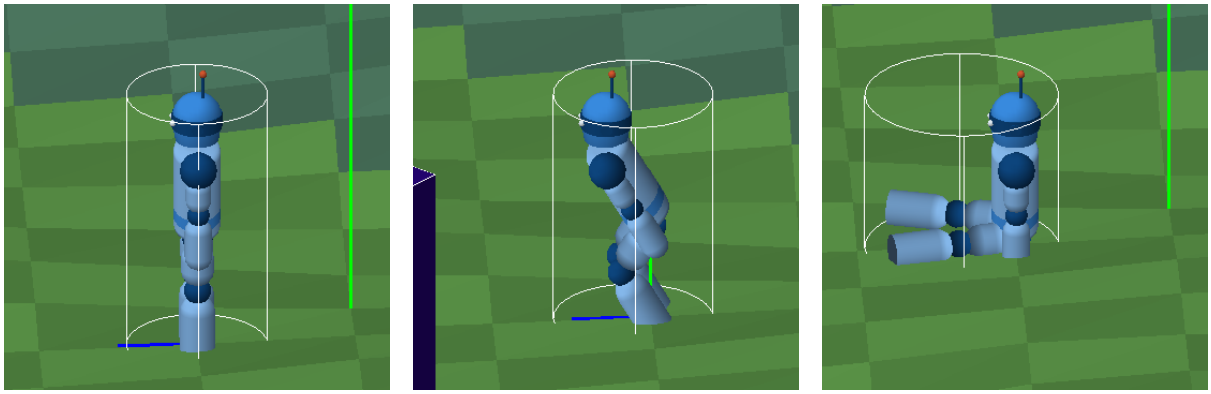
切換姿勢固定要花費 0.5 秒。轉換狀態記錄起點 `from`、終點 `to` 與進度 $t \in [0, 1]$，所有關節角度和 rootOffset 都依 t 線性內插。如果途中要求回到原姿勢，程式會交換 `from` 與 `to`，並將進度改成 $1 - t$，讓動作直接從目前位置倒回去。此時若要求第三種姿勢，輸入會先被忽略，等目前的轉換結束後才能再次切換。轉換期間不能移動，碰撞箱也會以相同進度改變大小（第 4 節）。

3.3 走路與跑步

走路動畫參考 Minecraft 的手腳擺動方式，動畫相位是由機器人累積水平移動距離 d 決定：

$$\phi = 2\pi \frac{d}{C}, \qquad C = 2 \cdot 2L \sin 25^\circ \approx 1.37 \text{ m},$$

其中 C 代表左右腳各跨一步的距離。大腿前後各擺 25° 時，兩腳在前後方向相距 $2L \sin 25^\circ$，所以一個完整週期取其兩倍。擺動幅度 a 的目標值為「目前速度／走路速



(a) 站立 (b) 潛行 (c) 坐下

Figure 3: 三種姿勢；線框為碰撞箱

度」，上限為 1.2，停下時則為 0。另一個係數 $r \in [0,1]$ 表示跑步程度；速度超過走路速度後開始增加，達到兩倍走路速度時為 1。各關節在站立姿勢上疊加的 pitch 角度為

$$\text{髖} = \pm 25^\circ \sin\phi \cdot a, \qquad \text{膝} = -40^\circ \max(0, \pm\cos\phi) \cdot a,$$
$$\text{肩} = \mp 20^\circ \sin\phi \cdot a + 180^\circ r, \qquad \text{肘} = 15^\circ \cdot a\,(1 - r),$$

其中 $\pm$、$\mp$ 的上方符號用於身體左側關節、下方用於身體右側關節，pitch 正值為往前抬，因此兩腳交替、手與同側的腳反向擺動。$\cos\phi$ 用來判斷大腿擺動方向，只有往前擺的那條腿會彎曲膝蓋，支撐腿保持伸直。當髖關節角度為 $\theta = 25^\circ \sin\phi \cdot a$ 時，rootOffset 會再扣掉 $L(1 - \cos\theta)$，使髖部隨步伐上下移動。

跑步時，雙臂會逐漸伸直並舉過頭，再向外張開 $20^\circ r$ 成 V 字。a 與 r 不會立刻跳到目標值，而是在每個 tick 以 $x \leftarrow x + (x^* - x)(1 - e^{-k\,\Delta t})$ 逼近。因此起跑與停下時，四肢會逐漸展開或回到原位。這個更新式不受 tick 頻率影響：連續更新兩次 Δt 與一次更新 $2\Delta t$ 的結果相同。

3.4 轉向與視線

視線由 yaw 與 pitch 組成，可用滑鼠拖曳。身體朝向直接使用視線的 yaw，移動方向也以此為基準；視線的 pitch 則交給頭部關節，超出活動範圍的部分會被截斷。切換到自由相機後，相機的旋轉不再改變機器人視線，機器人會維持原本的方向。

3.5 跳躍與重力

重力使用半隱式 Euler 法積分，每個 tick 先更新速度，再用新速度更新位置：

$$\mathbf{v} \leftarrow \mathbf{v} + \mathbf{a}\Delta t, \qquad \mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}\Delta t, \qquad \mathbf{a} = [0, -g, 0],\ g = 9.80665\ \text{m/s}^2.$$

著地時假設摩擦力足夠，水平速度可直接改成目標速度。離地後只受重力影響，因此會保留起跳瞬間的水平速度，不能在空中轉向或加速。設定跳躍高度 $h = 0.5$ m，再由 $v^2 = 2gh$ 算出鉛直起跳速度 $v_y = \sqrt{2gh}$。

3.6　動作限制

- **速度**：站立時走路 1.2 m/s、跑步 3 m/s，潛行 0.6 m/s，坐下或轉換姿勢時為 0；前後與左右同時按下時，合成方向的長度會被限制在 1，斜走不會比直走快。
- **關節角度**：每次計算骨架矩陣前，所有角度都會被截斷到表 2 的範圍。
- **姿勢**：只有站立能跑，坐下不能移動或跳躍。

4　碰撞偵測

4.1　碰撞形狀

機器人的碰撞箱採用直立圓柱，基準點位於腳底中心。障礙物分成可繞 y 軸旋轉的方塊、直立圓柱與球。除了球以外，所有碰撞形狀都維持直立，因此計算時可將鉛直與水平重疊分開處理，不需要做一般三維多面體碰撞。

圓柱會隨姿勢改變。站立時半徑為 0.45 m、高 1.75 m；潛行時高度降為 1.57 m，中心向前移 0.11 m；坐下後高度為 1.06 m，中心向前移 0.34 m。這些尺寸取自各姿勢實際占用的範圍。中心偏移會跟著身體朝向旋轉，切換姿勢時，尺寸和偏移也會一起內插。這樣潛行時可以進入較低的空間，坐下後向前伸出的腿也會包含在碰撞範圍內。圖 3 的線框即為三種姿勢的碰撞箱。

4.2　偵測與推擠

每個 tick 先依速度移動碰撞箱，再檢查周圍 3×3 個 chunk 中的障礙物。偵測函式會回傳接觸法線 $\mathbf{n}$ 與穿透深度 δ，三種形狀分別以下列方式計算：

- **方塊**：把圓柱中心轉到方塊的局部座標，將中心夾在方塊的 x、z 範圍內得到最近點，最近點到圓心的距離小於半徑即為碰撞。
- **圓柱**：兩圓心的水平距離小於半徑和即為碰撞。
- **球**：在圓柱上找離球心最近的點（高度夾在圓柱上下緣之間，水平方向夾在圓周內），距離小於球的半徑即為碰撞。

水平與鉛直重疊量算出後，選擇較小的一項作為推出方向。例如從方塊上方落下時，通常是鉛直重疊較小，機器人便會被向上推出並停在頂面。

發生碰撞時，碰撞箱沿 $-\mathbf{n}$ 推出 δ，並從速度移除朝向障礙物的分量 $(\mathbf{v} \cdot \mathbf{n})\,\mathbf{n}$，保留平行表面的速度。因此機器人斜撞牆面時仍可沿牆滑動，不會完全停下。一次推出可能造成另一處重疊，所以每個 tick 最多重複 5 次，或在沒有碰撞後提早停止。兩個表面剛好相切時仍算接觸，穿透深度記為 0；否則機器人站在方塊上時，著地狀態會在相鄰 tick 之間反覆切換。

4.3 陡坡與支撐面

地形由三角形組成，任一水平位置都能查到高度與法向量。當法向量和 y 軸的夾角小於 45°（$n_y > \frac{\sqrt{2}}{2}$）時，該表面可以行走。更陡的坡面視為牆壁；若腳底低於坡面，就沿法向量的水平分量向外推出，距離為高度差乘上 cot（坡度），使機器人無法繼續爬升。

完成推擠後，程式會在腳底下方搜尋支撐面，取可行走地形與朝上障礙物頂面中最高的一個。腳底碰到支撐面即判定為著地。另外，如果上一個 tick 已經著地、目前沒有向上跳，而且離支撐面不超過 0.3 m，機器人會直接貼回表面。這項處理是為了解決下坡時每一步都短暫離地，導致移動控制失效的問題。

5 世界與相機

5.1 地形與 Chunk

原本的作業規格使用 $[0, 0, 0]$ 到 $[50, 0, 50]$ 的平面。我將地板改成依 seed 生成的起伏地形，並切成邊長 16 m 的 chunk。程式只保留機器人周圍半徑 12 個 chunk，移動時會釋放遠處區塊並生成新區塊，因此地形可以持續向外延伸（圖 4）。

地形高度由自行實作的 Perlin noise 與 fBm 產生，再依高度與環境參數決定地表顏色。各 chunk 的內容由世界 seed 和座標決定，相同 seed 會得到相同結果。地形網格只在 chunk 第一次繪製時建立並快取，避免每幀重新取樣雜訊。

5.2 障礙物

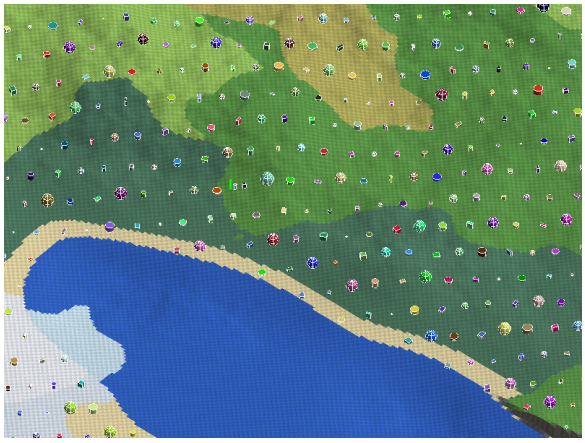
每個 chunk 預先選定三個位置，分別嘗試放置方塊、球與圓柱；若位置位於水面或冰面則不生成。方塊的長寬高與旋轉角度、球的半徑、圓柱的半徑與高度都由位置 seed 決定，尺寸上限為 2 m，顏色也使用同一 seed 產生。障礙物底部貼齊當地地面，並和地形一樣轉成網格繪製，再套用第 2 節的 Half-Lambert 明暗。

5.3 相機與投影

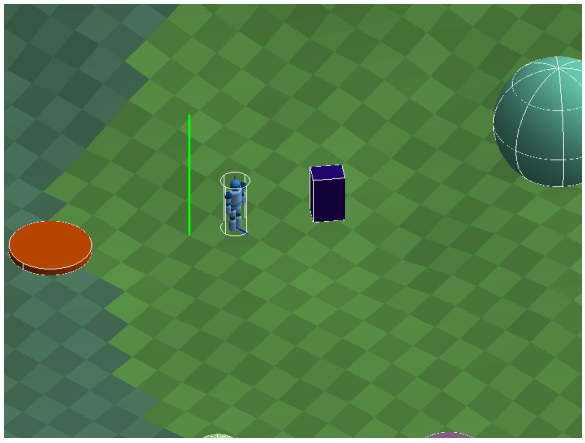
相機使用 eye、focus、up 三個參數，交給 `gluLookAt`；投影方式沿用作業給定的 `glOrtho` 平行投影。focus 取機器人眼睛的世界座標，計算方式見第 2.2 節。eye 則依相機的 yaw、pitch，從 focus 沿反方向移動一段距離：

$$\mathbf{f} = (-\sin\psi\cos\theta,\ \sin\theta,\ -\cos\psi\cos\theta), \qquad \text{eye} = \text{focus} - d\,\mathbf{f},$$

其中 ψ、θ 分別是 yaw 與 pitch，$\mathbf{f}$ 是向量 $(0, 0, -1)$ 先俯仰、再繞 y 軸旋轉後的結果。pitch 限制在 ±89.9°，避免視線方向與 up 平行。



(a) 地形與生態系概觀



(b) 方塊、球與圓柱障礙物

Figure 4: 地形與障礙物

`glOrtho` 的左右範圍會配合視窗長寬比調整，避免改變視窗大小時畫面變形。near 與 far 分別設為 −400 和 400，以免 eye 後方的高地或載入範圍邊緣被裁掉。每幀先繪製 3D 場景，接著切換成以像素為單位的 `glOrtho`，再畫出 HUD。

6 心得

這是我第一次設計需要隨時間連續更新狀態的即時互動程式。過去寫的程式主要是網頁服務，只要等待使用者操作或請求，再處理對應的事件；這次即使沒有輸入，機器人的動作、物理狀態和世界仍要在每個 tick 持續更新。剛開始時，我不太確定該怎麼劃分各部分的責任，原本習慣的事件處理方式很快就讓輸入、動畫和繪圖混在一起。

為了整理出合適的架構，我花了很多時間參考 Minecraft 等遊戲處理角色動作與世界更新的方式，再逐步拆分輸入、模擬和繪製。機器人的部分也經過數次調整，最後才將骨架、姿勢和外觀分開保存。這個過程不是一開始就規劃好的，中途改過很多次，有些部分甚至直接推翻重做。雖然重構花了不少時間，但當站立、潛行、走路、跑步和碰撞陸續加進來時，我才感覺架構開始能承受功能的增加。

這次最讓我有成就感的部分，是自己從零開始實作 Perlin noise。閱讀網路上的原理說明後，我依序完成格點梯度、內積、平滑內插與 seed 的處理，再將多層雜訊疊加成 fBm，用來生成連續的地形。看到相同 seed 能產生相同世界，而且分開生成的 chunk 邊界可以接在一起時，我覺得前面花的時間很值得。地形原本不是作業重點，卻是我投入最多時間的功能之一，也是這次最想保留下來的成果。